

Lab: Secure Multi-Layer IAM Architecture with Auditing

Objective: Build a production-grade secure environment demonstrating:

- IAM Users, Groups, Roles vs Users
- Custom IAM Policies & Least Privilege
- MFA Enforcement
- CloudTrail Auditing
- Security Best Practices

Region: af-south-1 (Cape Town)

Pre-Lab Preparation

1. Log in with your **IAM Admin user** (not Root)
2. Switch to **af-south-1**
3. Have your authenticator app ready for MFA

Phase 1: IAM Structure Setup

Step 1: Create IAM Groups

- Go to **IAM → Groups → Create group**
 - Group 1: Developers → Attach AmazonEC2FullAccess + AmazonS3ReadOnlyAccess

The screenshot shows the AWS IAM console interface. On the left is a navigation sidebar with 'Identity and Access Management (IAM)' selected. The main content area is titled 'Developers' and shows a 'Summary' card with the group name 'Developers', creation time 'May 18, 2026, 20:44 (UTC+01:00)', and ARN 'arn:aws:iam::[account-id]:group/Developers'. Below this, the 'Permissions' tab is active, showing 'Permissions policies (2)' attached to the group. A table lists the attached policies:

Policy name	Type	Attached entities
AmazonEC2FullAccess	AWS managed	1
AmazonS3ReadOnlyAccess	AWS managed	1

At the bottom of the console, the URL 'https://us-east-1.console.aws.amazon.com/console/home?region=us-east-1' and copyright information '© 2026, Amazon Web Services, Inc. or its affiliates.' are visible.

- Group 2: DevOps → Attach AdministratorAccess.

The screenshot shows the AWS IAM console interface. On the left is a navigation sidebar with sections: Identity and Access Management (IAM), Access Management, and Access reports. The main content area is titled 'DevOps' and includes a 'Summary' section with fields for User group name (DevOps), Creation time (May 18, 2026, 20:44 (UTC+01:00)), and ARN (arn:aws:iam::[redacted]:group/DevOps). Below the summary are tabs for Users (1), Permissions, and Access Advisor. The 'Permissions' tab is active, showing 'Permissions policies (1)' with a search bar and a 'Filter by Type' dropdown set to 'All types'. A table lists the attached policies:

Policy name	Type	Attached entities
AdministratorAccess	AWS managed - job function	2

At the bottom of the console, there is a footer with 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc. or its affiliates.

Step 2: Create IAM Users

- Create 2 users:
 - emeka-dev → Add to **Developers** group

The screenshot shows the AWS IAM console interface for the 'emeka-dev' user group. The left sidebar is identical to the previous screenshot. The main content area is titled 'emeka-dev' and includes a 'Summary' section with fields for ARN (arn:aws:iam::[redacted]:user/emeka-dev), Console access (Enabled with MFA), Access key 1 (Create access key), Created (May 18, 2026, 20:47 (UTC+01:00)), and Last console sign-in (Never). Below the summary are tabs for Permissions, Groups (1), Tags, Security credentials, and Last Accessed. The 'Groups (1)' tab is active, showing 'User groups membership' with a search bar and a table listing the groups the user belongs to:

Group name	Attached policies
Developers	AmazonEC2FullAccess and AmazonS3ReadOnlyAccess

At the bottom of the console, there is a footer with 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc. or its affiliates.

- ada-devops → Add to **DevOps** group

The screenshot shows the AWS IAM console interface. On the left is the navigation menu with sections like 'Identity and Access Management (IAM)', 'Access Management', and 'Access reports'. The main content area displays the details for the 'ada-devops' user. The 'Summary' section shows the user's ARN, creation date, and console access status. The 'Groups' tab is active, showing that the user is a member of the 'DevOps' group, which is associated with the 'AdministratorAccess' policy. There are buttons for 'Delete', 'Add user to groups', and 'Remove'.

- Enable **Console access** for both + require password reset

Step 3: Create a Custom Least Privilege Policy

- IAM → Policies → Create policy (JSON tab)
- Name: Week2-DeveloperPolicy

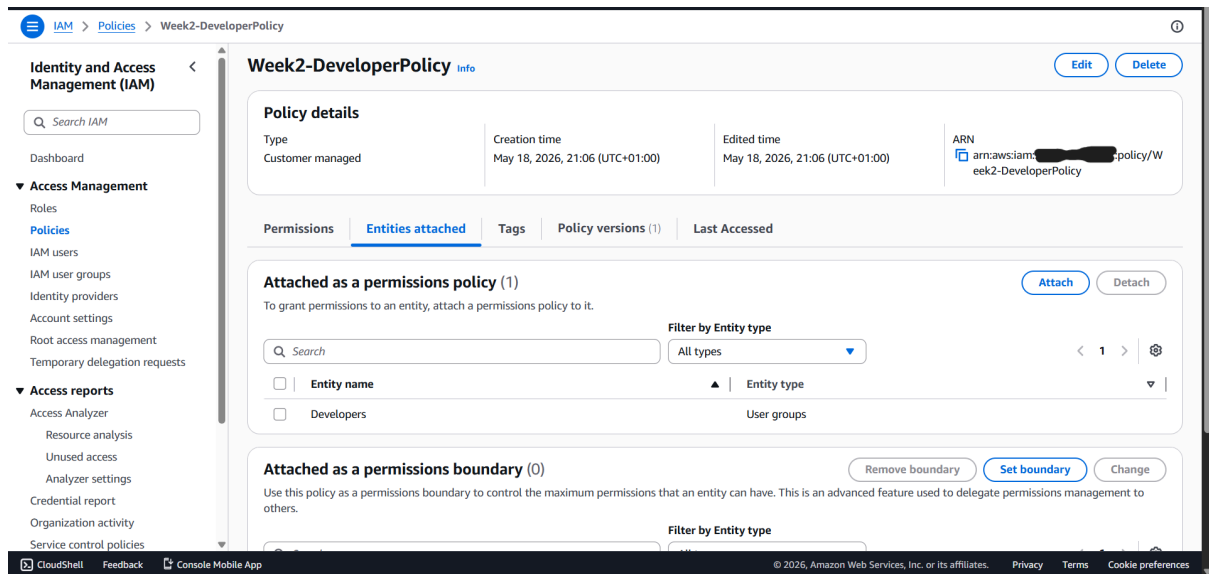
Week2-DeveloperPolicy

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "s3:ListBucket",
8          "s3:GetObject"
9        ],
10       "Resource": [
11         "arn:aws:s3:::week2-capstone-bucket",
12         "arn:aws:s3:::week2-capstone-bucket/*"
13       ]
14     },
15     {
16       "Effect": "Allow",
17       "Action": "ec2:Describe*",
18       "Resource": "*"
19     }
20   ]

```

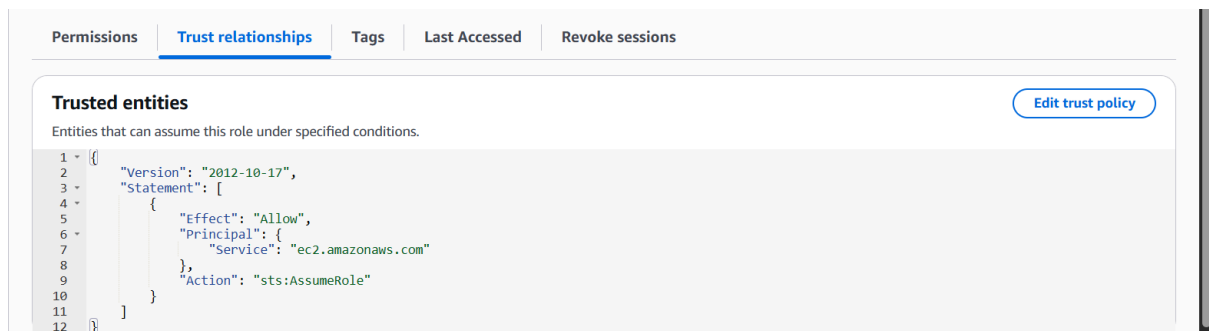
- Attach this policy to the **Developers** group (in addition to the managed ones)



Phase 2: IAM Roles & Service Access

Step 4: Create EC2 Instance Role

- IAM → Roles → Create role
 - Trusted entity: AWS service → EC2



- Permissions: Attach Week2-DeveloperPolicy + AmazonSSMManagedInstanceCore

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

Filter by Type: All types

☐	Policy name	Type	Attached entities
☐	AmazonSSMManagedInstanceCore	AWS managed	1
☐	Week2-DeveloperPolicy	Customer managed	2

- Role name: Week2-EC2AppRole

Week2-EC2AppRole [Info](#) [Delete](#)

Allows EC2 instances to call AWS services on your behalf.

Summary [Edit](#)

Creation date: May 18, 2026, 21:19 (UTC+01:00)

ARN: [arn:aws:iam::\[redacted\]:role/Week2-EC2AppRole](#)

Instance profile ARN: [arn:aws:iam::441870953802:instance-profile/Week2-EC2AppRole](#)

Last activity: -

Maximum session duration: 1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) [Info](#)

You can attach up to 10 managed policies.

Filter by Type: All types

☐	Policy name	Type	Attached entities
☐	AmazonSSMManagedInstanceCore	AWS managed	1
☐	Week2-DeveloperPolicy	Customer managed	2

Step 5: Launch EC2 with the Role

- Launch an EC2 instance (t3.micro, Amazon Linux 2023) in af-south-1
- In **Advanced settings** → IAM instance profile → select Week2-EC2AppRole
- Security Group: Allow SSH from your IP + HTTP from anywhere


```
ec2-user@ip-172-31-8-225:~$ ssh -i "Week2-EC2AppRole.pem" ec2-user@ec2-13-247-182-70.af-south-1.compute.amazonaws.com
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Mon May 18 20:35:12 2026 from 102.89.82.113
[ec2-user@ip-172-31-8-225 ~]$ aws s3 ls s3://week2-capstone-bucket
An error occurred (NoSuchBucket) when calling the ListObjectsV2 operation: The specified bucket does not exist
[ec2-user@ip-172-31-8-225 ~]$ aws s3 mb s3://week2-capstone-bucket --region af-south-1
make_bucket failed: s3://week2-capstone-bucket An error occurred (AccessDenied) when calling the CreateBucket operation: User: arn:aws:sts::441870953802:assumed-role/Week2-EC2AppRole/1-03344e37cd618c477 is not authorized to perform: s3:CreateBucket on resource: "arn:aws:s3:::week2-capstone-bucket" because no identity-based policy allows the s3:CreateBucket action
```

Important Note for Learning

- s3:ListAllMyBuckets = List every bucket (broad permission)
- s3:ListBucket = List contents of **one specific** bucket
- no identity-based policy allows the s3:CreateBucket action

This is exactly why **Least Privilege** is important - and why this error is great for learning.

Phase 3: MFA Enforcement

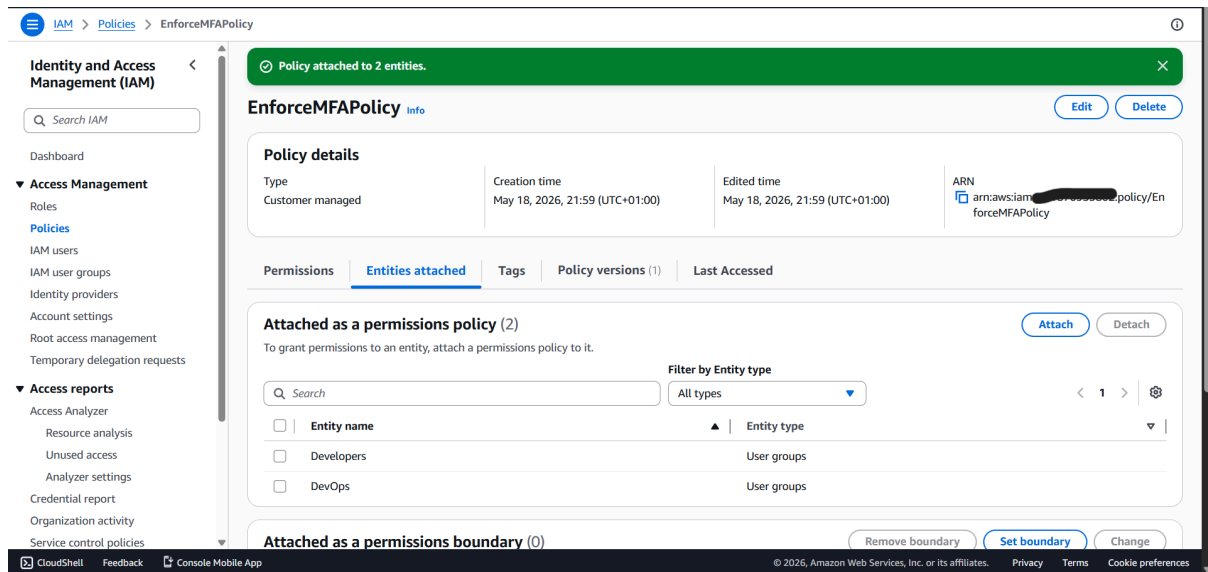
Step 6: Enforce MFA

1. Enable MFA on both emeka-dev and ada-devops
2. Create MFA Enforcement Policy (JSON):

JSON

```
EnforceMFAPolicy
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "DenyWithoutMFA",
6        "Effect": "Deny",
7        "Action": "*",
8        "Resource": "*",
9        "Condition": {
10       "BoolIfExists": {
11         "aws:MultiFactorAuthPresent": "false"
12       }
13     }
14   ]
15 }
16 }
```

- Name: EnforceMFAPolicy
- Attach to both users (or the groups)



Test: Log out and log in again, you should be forced to use MFA.

Phase 4: CloudTrail Auditing

Step 7: Enable CloudTrail

- Go to **CloudTrail** → **Trails** → **Create trail**
 - Trail name: Week2-Security-Trail
 - Apply trail to all regions: **Yes**
 - Create new S3 bucket: week2-cloudtrail-logs-<yourname>
 - Enable **Log file integrity validation**
 - Enable **CloudWatch Logs** (optional but recommended)

Trails [Info](#)

Copy events to Lake

Create trail

Name	▲	Status
Week2-Security-Trail		✔ Logging

Wait 10–15 minutes, then perform some actions (create/delete a test S3 bucket) and check the logs.

Buckets

General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (1) [Info](#)

↺

Copy ARN

Empty

Delete

Create bucket

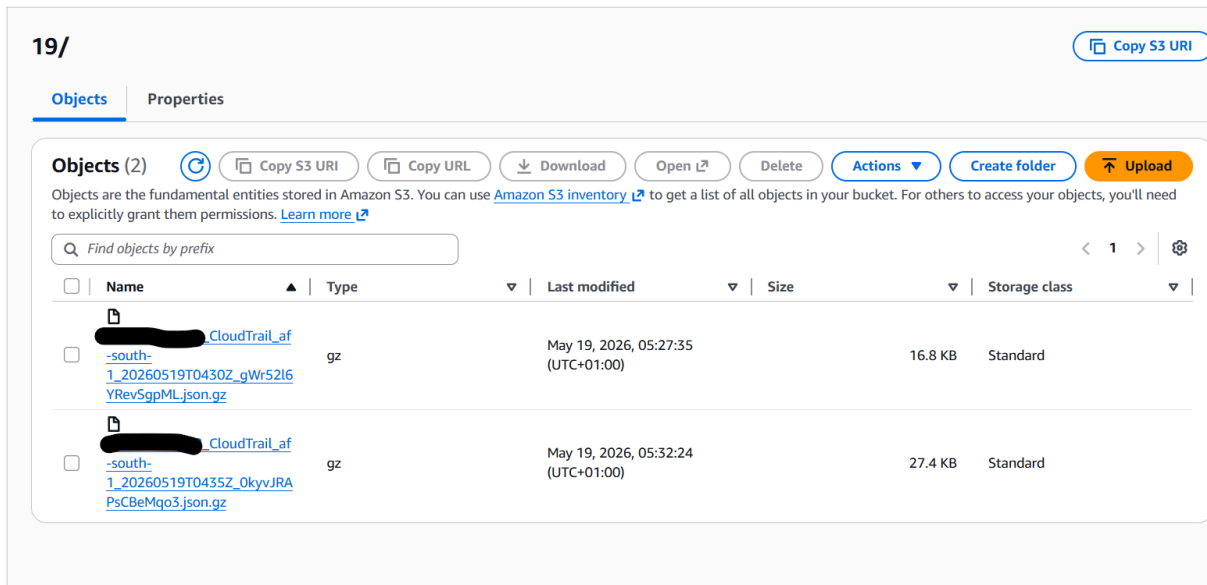
Buckets are containers for data stored in S3.

🔍 Find buckets by name

< 1 >

⚙️

	Name	▲	AWS Region	▼	Creation date	▼
☐	aws-cloudtrail-logs- [REDACTED]-58df7a1a		Africa (Cape Town) af-south-1		May 19, 2026, 05:25:03 (UTC+01:00)	



Phase 5: Final Verification & Cleanup

Verification Checklist

1. IAM Users + Groups with correct policies
2. Custom IAM Policy JSON
3. EC2 instance with attached Role (and successful aws CLI commands)
4. MFA enabled + Enforcement policy attached
5. CloudTrail trail active with Log file integrity enabled
6. Billing Alarm still active

Cleanup:

- Stop/Terminate the EC2 instance
- (Optional) Delete the CloudTrail trail and S3 bucket if you want to be extra clean

Thank You